

**[Condominio “El Mirador”]
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	Documento Arquitectura de Software Versión 1.0
Proyecto	Condominio "El Mirador"
Versión	1.0
Documento mantenido por	Matias Durán - Pedro Suarez - Claudio Muñoz
Fecha de última revisión	05 de Abril
Fecha de próxima revisión	10 de Abril
Documento aprobado por	Matias Durán - Pedro Suarez - Claudio Muñoz
Fecha de última aprobación	09 de Abril

Historia de Revisiones

Fecha	Versión	Descripción	Autor
05/04/2026	1.0	Inicio proyecto	Pedro Suarez Claudio Muñoz Matias Duran

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

Actualmente, la administración del edificio “El Mirador” se realiza de manera manual o con herramientas poco integradas, lo que está generando dificultades en el control de residentes, pagos y solicitudes. Esto provoca errores en la gestión, falta de información oportuna, retrasos en los pagos y poca transparencia en los procesos administrativos.

1.2. Propósito

El propósito de este proyecto es desarrollar un sistema de gestión que permita automatizar los procesos administrativos del edificio, mejorando la eficiencia, el control de la información y la toma de decisiones por parte del administrador.

1.3. Ámbito

El sistema abarca las siguientes funcionalidades:

- Gestión de residentes
- Administración de pagos y morosidad
- Registro de solicitudes y reclamos
- Generación de reportes

El sistema será utilizado por administradores, conserjes y residentes, mediante una plataforma digital accesible

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>RF</i>	Requisito Funcional
<i>RNF</i>	Requisito No Funcional
<i>CRUD</i>	Crear, Leer, Actualizar y Eliminar datos
<i>UI</i>	Interfaz de Usuario
<i>BD</i>	Base de Datos
<i>API</i>	Interfaz de Programación de Aplicaciones
<i>ERP</i>	Sistema de Planificación de Recursos Empresariales

1.5. Resumen ejecutivo

El presente proyecto propone el desarrollo de un sistema de gestión para el edificio “El Mirador”, enfocado en automatizar procesos clave como la administración de residentes, control de pagos y generación de reportes.

La solución permitirá mejorar la eficiencia operativa, reducir errores humanos, aumentar la transparencia y facilitar la gestión administrativa. Además, considera requisitos funcionales y no funcionales que aseguran la calidad, seguridad y rendimiento del sistema.

1.6. Arquitectura del sistema

4+1) Se propone una arquitectura basada en el modelo de múltiples vistas (modelo

Vista de Escenario (Casos de uso)

Este describe cómo los usuarios interactúan con el sistema.

El Administrador gestiona los residentes y pagos

El Residente consulta información y realiza solicitudes

Sistema envía notificaciones automáticas

Vista Lógica

Muestra la estructura funcional del sistema

- Módulo de gestión de residentes
- Módulo de pagos
- Módulo de reportes
- Módulo de solicitudes

Vista de Desarrollo

Describe cómo está construido el sistema a nivel técnico

-Frontend (interfaz de usuario)

-Backend (lógica del negocio)

-Base de datos

Vista de Proceso

Muestra cómo se comporta el sistema en ejecución

- Procesos concurrentes (usuarios simultáneos)
- Notificaciones automáticas en segundo plano
- Manejo de solicitudes en tiempo real

Vista Física (Despliegue)

Representa donde se ejecuta el sistema

-Servidor web

-Servidor de base de datos

- Acceso desde navegadores (PC o móvil)

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema de gestión para el Condominio “El Mirador” es una plataforma digital integral diseñada para centralizar, automatizar y optimizar las operaciones administrativas y de comunicación dentro del edificio. Esta solución surge como respuesta a las ineficiencias de la gestión manual actual, buscando eliminar la fragmentación de la información y los errores en los procesos de cobro.

La arquitectura del sistema se basa en el modelo de múltiples vistas, lo que permite una estructura robusta y escalable que integra diversos módulos críticos los cuales son:

- **Gestión de Residentes:** Un repositorio centralizado para el registro y actualización de datos de propietarios y arrendatarios.
- **Administración Financiera:** Automatización del cálculo de gastos comunes (basado en consumos y remuneraciones), control de pagos y detección de morosidad.
- **Módulo de Comunicación y Servicios:** Un canal formal para que los residentes ingresen solicitudes de mantención o reclamos, permitiendo un seguimiento transparente por parte de la administración.
- **Generación de Reportes:** Herramientas de análisis para obtener informes oportunos sobre el estado financiero y operativo del condominio.

El sistema opera bajo un esquema de control de acceso basado en roles, asegurando que administradores, conserjes y residentes interactúen con una interfaz intuitiva y segura, adaptada a sus necesidades específicas. En definitiva, la plataforma actúa como el núcleo operativo del edificio, garantizando transparencia, rapidez en la respuesta y una base de datos confiable para la toma de decisiones.

2.2. Objetivos del sistema

El desarrollo del sistema para el Condominio “El Mirador” busca transformar la gestión operativa actual mediante el cumplimiento de los siguientes objetivos fundamentales:

Objetivo General

- Automatizar y centralizar los procesos administrativos del edificio para mejorar la eficiencia operativa, el control de la información y la transparencia en la toma de decisiones del administrador.

Objetivos Específicos

- Optimizar la gestión financiera: Implementar un sistema de cálculo automatizado para los gastos comunes y la identificación de morosidad, reduciendo así las pérdidas económicas históricas del condominio.
- Reducir el error humano: Sustituir los registros manuales por una base de datos centralizada que minimice la duplicidad de información y los retrasos en los procesos de cobro.
- Fortalecer la comunicación: Proveer un canal formal y transparente para que los residentes realicen solicitudes y reclamos, permitiendo un seguimiento en tiempo real por parte de la administración.
- Garantizar la disponibilidad y seguridad: Asegurar que la información financiera y de los residentes esté protegida mediante control de acceso por roles y copias de seguridad constantes.
- Facilitar el análisis administrativo: Generar reportes e informes oportunos sobre el estado de pagos, residentes y deudas para respaldar una gestión basada en datos reales.

2.3. Requerimientos funcionales

Nombre requerimiento	Descripción Corta
Registro de Residentes	Permite registrar información de propietarios, arrendatarios, y personal.
Actualización de datos de residentes	Permite modificar información de residentes existentes
Consulta de residentes	Permite visualizar listado y detalles de residentes.
Calculo automatizado de gastos	Automatización del cálculo de cuotas basado en consumos (agua, luz, gas) y remuneraciones del personal.

Generación de recibos	Genera comprobantes de pago.
Notificación de pagos	Envía recordatorios de pago automáticamente.
Gestión de morosidad	Identifica residentes con deudas pendientes.
Registro de solicitudes	Permite ingresar solicitudes de mantención o servicios.
Gestión de quejas	Permite registrar y gestionar reclamos.
Generación de informes	Permite generar reportes de pagos, residentes, morosidad

2.4. Requerimientos no funcionales

Nombre requerimiento	Descripción Corta
Seguridad del sistema	Protección de datos y acceso seguro (Acceso mediante autenticación, datos cifrados)
Usabilidad	Sistema fácil de usar e intuitivo (Interfaz clara, navegación simple)
Disponibilidad	Acceso continuo al sistema.
Control de Acceso y Privacidad	Restricción de acceso a la información financiera según el perfil del usuario (Admin vs Residente).
Rendimiento	El sistema debe responder rápidamente a las solicitudes de los usuarios.
Escalabilidad	El sistema debe poder crecer según aumente la cantidad de usuarios o datos.

Mantenibilidad	El sistema debe ser fácil de mantener y actualizar.
Respaldo de información	El sistema debe realizar copias de seguridad de los datos

2.5. Supuestos y dependencias:

Dentro de todo el sistema, el mayor enfoque con los requisitos funcionales, es marcado hacia los registros de clientes e informes de pago, siendo estos esenciales para mejorar y automatizar el sistema que tanto ha pedido tal cambio.

Siendo este cambio el cobro y notificación por morosidad de la gente, un elemento que suele causar pérdidas para “El Mirador” , una actualización más que necesaria para el sistema; claramente siendo esta la prioridad en el enfoque que le hemos dado al proyecto. De esta manera podríamos dar un orden dentro del trabajo que se ha de realizar.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

3.2. Justificación del estilo según el contexto del sistema

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

4.1.2. Actores

4.1.3. Diagrama general de casos de uso

4.1.4. Diagrama de casos de uso específicos

4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	
CU-002-001	Exportar actividades	

4.1.6. s p e c i f i c a c i ón de casos de uso	CU-002-002	Importar deuda vencida por PDV	
	CU-004-001	Generación Archivo PDA Importación	

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]	
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]	
Descripción	[Descripción del caso de uso]	
Resumen	[Resumen de alto nivel del funcionamiento]	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]
[Se describe el proceso o secuencia de pasos ejecutadas usando frases cortas] [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
[Número o de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

4.2.2. Diagrama de clases

4.2.3. Descripción diagrama de clases

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

- 4.3.1. Propósito
- 4.3.2. Diagrama de componente
- 4.3.3. Descripción diagrama de componente
- 4.3.4. Diagrama de paquete
- 4.3.5. Descripción diagrama de paquete

4.4. VISTA DE PROCESOS

- 4.4.1. Propósito
- 4.4.2. Diagrama de actividad
- 4.4.3. Descripción diagrama de actividad

4.5. VISTA FÍSICA

- 4.5.1. Propósito
- 4.5.2. Diagrama de despliegue
- 4.5.3. Descripción diagrama de despliegue

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCION	APLICACION EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt

[illegible]

[illegible]